601.445/601.645: Practical Cryptographic Systems November 9, 2026

Assignment 2, Parts 2 and 3: break it

Instructor: Matthew Green *Due: 11:59 pm, November 20*

This is the kit's `EXERCISES.md`*, typeset. The kit itself, with* `API.md`*,* `REVIEW-LAB.md`*, and the* `AGENTS.md` *addendum, is at* `https://github.com/matthewdgreen/practicalcrypto2026/blob/main/assignments/A2/exercise-kit.zip`

This is the exercise kit the handout promised. Parts 0, 1 and 4 are in the main handout; read this together with it and with the protocol specification. Parts 2 and 3 are **graded on your final submission** (Friday, November 20, 11:59 pm).

You have already built a JMessage client that faithfully implements a protocol with some deliberately bad decisions. Now you play the one party the protocol treats as untrusted — **the server** — and show that two of those decisions let that party do real harm.

Everything here runs locally against a sandbox we supply. **Do not point any of this at the course server, the `echo` account, or another student's client.** The handout's integrity rules still apply.

What you are attacking

The sandbox runs two honest clients, `alice` and `bob`. They use the released reference implementation, with authenticated keys from a fixture, so directory substitution is not a way in. You sit between them as the relay: you can read, withhold, reorder, duplicate, drop, and inject protocol messages with either name on them. You cannot read a bot's memory, keys, or secret, set its compatibility option, or ask it to decrypt for you. Your only signals are the wire messages the bots emit and the limited status the API reports.

Alice sends **one secret message** to Bob, once, as the first record of her session. Your job is to defeat the protocol well enough to read it.

Part 2 — Downgrade the negotiation (negotiation mode)

Alice offers both suites, preferring the hybrid JM2. An unchanged relay lets the two bots establish JM2, and you cannot read JM2 records. **Make the two honest bots establish the legacy JM1 suite instead**, without either of them detecting anything wrong.

The handshake's transcript check already stops the obvious move. Try it: the `experiment` entry point (below) lets you strip JM2 from Alice's offer and watch the handshake fail at the responder's transcript verification. Your write-up must include that experiment and explain *why* it fails. The downgrade that works does not modify Alice's offer; it uses a different part of the protocol. Read specification §§6.4–6.8 closely, and think about what each signature actually covers.

Write `downgrade(api)` in `exercises/attack.py`. It receives a started negotiation sandbox with Alice's HELLO already queued, and must return the 16-byte `sid` of the JM1 session both bots establish. For a chained run, leave Alice's secret record undelivered to Bob.

Part 3 — Read the legacy record (record mode, or after your downgrade)

Once a JM1 session exists, the legacy record layer leaks. JM1 checks a record's padding before it checks the MAC, and it reacts differently to the two failures (specification §7.2). That difference is an oracle. Use it to recover Alice's secret from her captured record, one byte at a time.

Record mode hands you a sandbox where a JM1 session is already established and Alice's secret record is captured and withheld, so you can work on Part 3 without first solving Part 2. The chained run uses the session your own downgrade produced.

Write `recover(api, target)`. It receives the sandbox and the captured DATA envelope, and returns:

```
{"plaintext": <bytes or None>, "blocks": {block_index: <16 bytes>, ...}}
```

`plaintext` is Alice's message, without the MAC or padding. `blocks` is optional partial-progress evidence: recovered 16-byte blocks of the CBC-decrypted padded string, zero-indexed in ciphertext order (the IV is not block 0).

Write `recover_byte` yourself. The one step that turns a confirmed oracle answer into a plaintext byte is yours to write by hand, with a short comment deriving it from CBC decryption (handout Revision 3, "Write this yourself"). A coding agent may explain the padding oracle and review your draft, but may not write or correct `recover_byte`. Your `recover` must obtain every byte through it. Say in `ai-usage.md` whether you wrote it.

Running your attacks

Copy the kit's `exercises/` directory into your client directory (the one holding `protocol.py` and `checkpoint/`); that directory is your submission, and `exercises/attack.py` inside it is what you edit. Then, from the extracted kit, with `<dir>` your client directory:

```
python check_environment.py
python run_exercises.py --submission <dir> --part 2 --output <dir>/run-part2
python run_exercises.py --submission <dir> --part 3 --output <dir>/run-part3
python run_exercises.py --submission <dir> \
    --part chain  --output <dir>/run-chain
python run_exercises.py --submission <dir> \
    --part experiment --output <dir>/run-exp
```

Each run starts a **fresh** sandbox with a new secret and new keys, calls `start` for you, then runs your entry point in a separate process under a time limit. An output directory must not already exist. Each run writes `result.json`, a `transport-trace.jsonl`, and the captured worker output. Put every `--output` inside your client directory as a top-level `run-*` directory: `package_final.py` collects run evidence only from there, and your write-up cites those files. An unfinished entry point may raise `NotImplementedError`; the other part still runs. A spent budget is reported as `limit` in `result.json`, with the counts, not as a crash.

Limits (per fresh run)

Resource	Limit
Calls delivering a body to a bot	65,536
HTTP requests (including queue reads and drops)	200,000
Pending outgoing messages	1,024
`downgrade` wall-clock	60 s
`recover` wall-clock	300 s
chained run wall-clock	360 s

Requests are sequential; the sandbox serializes everything, so batching buys nothing. An ordinary byte-at-a-time recovery of a secret this size fits these limits with room to spare. There is no endpoint that checks or reveals the secret; the runner compares your recovered bytes to the truth once, after your attack exits.

What to submit

Fill in `exercises/README.md` (there is a template). Across Parts 2 and 3 it must give:

- how to run your entry points and which parts are complete;
- **Part 2:** Alice's original offer, the interventions you made, and the evidence that both bots reached JM1; plus the offer-stripping experiment and why it fails at the responder-transcript check;
- **Part 3:** the observable distinction you used, a few representative probes and their responses, your recovered output or block progress, the query count, which sequence number you used and why it stays usable, and how you resolved the last-byte padding ambiguity;
- the security property that failed, your attack's assumptions, and the limits of your evidence.

If you built an interface, bot, or agent for Part 1, say whether it would have shown you the downgrade; mark it not applicable if you built none.

Submit with `python package_final.py --submission <dir> --output final.zip`, run from the kit directory. This bundles your client, the preserved `checkpoint/`, `exercises/`, and the `run-*/` evidence at the top of `<dir>`; it leaves out the staff kit, credentials, and anonymous-report material. Inspect the archive before you submit it.

Grading

Parts 2 and 3 are 20 points each (handout §4), split about evenly between execution and explanation.

- **Execution (about 10 points per part)** is the runner's verdict on fresh staff instances, never your local `result.json`: for Part 2, both bots on JM1 in the challenge lineage; for Part 3, exact recovery of the secret, with partial credit for correctly recovered blocks reported in `blocks`. The chained run is an extra demonstration, not a requirement for either part.

- **Explanation (about 10 points per part)** is `exercises/README.md` against the list above. A clear account of a near-miss earns partial credit; a working attack you cannot explain does not earn full marks. The offer-stripping experiment and why it fails at the responder-transcript check are part of the Part 2 explanation.

The repair analysis belongs in Part 4's `written.md`; you may cross-reference it rather than repeat it. The review lab is graded separately (`REVIEW-LAB.md`).